

Multi-Level Page Tables Assignment

Due date (via moodle): **May 22th, 23:59**

Individual work policy

The work you submit in this course is required to be the result of your individual effort only. You may discuss concepts and ideas with others, but **you must program individually.** **You should never observe another student's code**, from this or previous semesters.

Students violating this policy will receive a **course grade of 250** (“did not complete course requirements”).

AI Usage Policy

AI tools (ChatGPT, Claude, Copilot, etc.) may be used for learning concepts, but the course material is your primary resource.

You must understand and be able to explain every line of code you submit. Submitting code you don't understand violates the individual work policy.

Required documentation: If you use AI tools, add a comment at the end of your `pt.c` file with a link to your conversation (e.g., `// AI chat: https://...`).

1 Introduction

The goal in this assignment is to implement simulated OS code that handles a multi-level (trie-based) page table that defines the address space of some process. You will implement two functions. The first function creates/destroys virtual memory mappings in a page table. The second function checks if an address is mapped in a page table. (The second function is needed if the OS wants to figure out which physical address a process virtual address maps to.)

Your code will be a simulation because it will run in a normal process. We provide two files, `os.c` and `os.h`, which contain helper functions that simulate some OS functionality that your code will need to call. For your convenience, there's also a `main()` function demonstrating usage of the code. However, the provided `main()` only exercises your code in trivial ways. We recommend that you change it to thoroughly test the functions that you implemented.

1.1 Target hardware

Our simulated OS targets an imaginary 64-bit x86-like CPU. When talking about addresses (virtual or physical), we refer to the least significant bit as bit 0 and to the most significant bit as bit 63.

Virtual addresses The virtual address size of our hardware is 64 bits, of which only the lower **57** bits are used for translation. The top 7 bits are guaranteed to be identical to bit 56, i.e., they are either all ones or all zeroes. The following depicts the virtual address layout:

1. A function to create/destroy virtual memory mappings in a page table:

```
void page_table_update(uint64_t pt, uint64_t vpn, uint64_t ppn);
```

This function takes the following arguments:

- (a) **pt**: The **physical page number** of the page table root (this is the physical page that the page table base register in the CPU state will point to). You can assume that **pt** has been previously returned by `alloc_page_frame()`.
- (b) **vpn**: The **virtual page number** the caller wishes to map/unmap.
- (c) **ppn**: Can be one of two cases. If **ppn** is equal to a special `NO_MAPPING` value (defined in `os.h`), then **vpn**'s mapping (if it exists) should be destroyed. Otherwise, **ppn** specifies the **physical page number** that **vpn** should be mapped to.

2. A function to query the mapping of a **virtual page number** in a page table:

```
uint64_t page_table_query(uint64_t pt, uint64_t vpn);
```

This function returns the **physical page number** that **vpn** is mapped to, or `NO_MAPPING` if no mapping exists. The meaning of the **pt** argument is the same as with `page_table_update()`.

You can implement helper functions for your code, but make sure to implement them in your `pt.c` file. You may not submit additional files (not even header files).

IMPORTANT: A page table node should be treated as an array of `uint64_t`s.

2.1 Something to think about (no need to submit)

How would your code change if you were required to free a page table node once all of the PTEs it contains become invalid? How would you detect this condition? How efficient would your approach be (i.e., how much overhead would it add on every page table update)?

3 Submission instructions

1. Submit just your `pt.c` file. (We will test it with our own `main` function.)
2. The program must compile cleanly (no errors or warnings) when the following command is run in a directory containing the source code files:

```
gcc -O3 -Wall -std=c11 os.c pt.c
```